Homework 2

Total Points: 100 (Correctness and Style)

Due: Monday, April 14th, 11:59 pm

Starter Files

Download hw02.zip. Inside the archive, you will find starter files for the questions of this homework. You cannot import anything to solve the problems.

IMPORTANT: Coding and Docstring Style

This is a reminder that your code style will be graded. Here are a few useful links:

4.2Style Guide on Course Website

Style Guide Examples

Submission

By the end of this homework, you should have submitted the homework via Gradescope. You may submit more than once before the deadline; only the final submission will be graded. Refer to Lab00 directions for submission.

Testing

At any point of the homework, use the following command to test your work:

```
>>> python3 -m doctest hw02.py
```

General Notes and Requirements

1. **DO NOT IMPORT ANY PACKAGES.**
2. For all questions, only basic loops (for/while) and/or conditional statements (if/elif/else), NO list comprehension (will be covered in future lectures). Other built-in methods are allowed.
3. Please add your own doctests (**at least three**) as the given doctests are not sufficient. **You will be graded on the doctests**.

4. For file handling questions, you should create new files in the files folder for your own doctests, and submit them along with your code file and given files. These files can be created manually using a notepad. You don't need to use code to generate them.
5. In this assignment you can assume that the input given to you is valid and does not have any errors. **No input validation required for HW02.**
6. Read the **Submission** section at the end closely before you submit to Gradescope.

Required Questions

Question 1

It turns out that many people on your team use preferred names.

Write a function that takes two lists: one containing given names (as strings) and another containing corresponding preferred names (also as strings).

Your function should return a list of tuples, where each tuple consists of the given name as the first element and the corresponding preferred name as the second element, in the same order as the input lists. If no preferred names are provided, the function should return an empty list.

Notes:

- You can assume that the list of given names is **at most as long** as the list of preferred names.
- If there are fewer given names than preferred names, use 'NO NAME PROVIDED' as a placeholder for the missing given name. (See the doctests for examples.)
- To create a tuple, use parentheses `()` instead of square brackets `[]`. For example: `var = (1, 2)`. Although tuples don't have an `append` method, you can access elements similarly by using an index, e.g., `var[1]`.

```
def name_mapping(given_names, preferred_names):
    """
    >>> given_names = ['Amanda', 'Jeffrey', 'Richard']
    >>> preferred_names = ['Mandy', 'Jeff', 'Rick']
    >>> name_mapping(given_names, preferred_names)
    [('Amanda', 'Mandy'), ('Jeffrey', 'Jeff'), ('Richard', 'Rick')]
```

```
    >>> given_names = ['Amanda', 'Jeffrey']
    >>> preferred_names = ['Mandy', 'Jeff', 'Rick']
    >>> name_mapping(given_names, preferred_names)
    [('Amanda', 'Mandy'), ('Jeffrey', 'Jeff'), ('NO NAME PROVIDED',
'Rick')]

    >>> given_names = []
    >>> preferred_names = ['Mandy', 'Jeff', 'Rick']
    >>> name_mapping(given_names, preferred_names)
    [('NO NAME PROVIDED', 'Mandy'), ('NO NAME PROVIDED', 'Jeff'),\
('NO NAME PROVIDED', 'Rick')]
    """
```

Question 2

Noticing how many people have preferred names, your boss has asked you to organize this data in a dictionary. However, before doing so, you should practice using this data structure a bit more.

Write a function that takes in two lists (of the same size), which may contain data of various types. In particular, you can expect:

- Integers and floats
- Strings (including empty ones)
- Lists and tuples (including empty ones)
- Dictionaries (including empty ones)
- Booleans

Your function should return a list of tuples, where each tuple contains corresponding elements from the two input lists in the same order. However, you should only keep the items from the first list if their type is valid as a dictionary key. If the type is not valid, replace that element with the string 'not valid' in the tuple.
Return an empty list if the input lists are empty (if one is empty, both will be empty, since input lists are the same size).

For example:

Input	Output	Explanation
keys = ["fun", ["not so much"]] values = [("learning",), 6]	[('fun', ('learning',)), ('not valid',)]	The first two items from both lists are “fun” and (“‘learning”,). Since strings can be dictionary keys (they are immutable), we create a tuple with these items and append it to the list. The second pair [“no so much”] and 6 can’t be added, since lists are mutable and can’t be used as a dictionary’s key.

```
def valid_pairs(keys, values):
    """
    >>> keys = ["fun", ["not so much"]]
    >>> values = [("learning",), 6]
    >>> valid_pairs(keys, values)
    [('fun', ('learning',)), ('not valid',)]

    >>> keys = [1, "fun", [2], (1,), {}]
    >>> values = [1, {}, (1,), "island", [2]]
    >>> valid_pairs(keys, values)
    [(1, 1), ('fun', {}), ('not valid',), ((1,), 'island'), ('not
valid',)]

    >>> keys =[]
    >>> values =[]
    >>> valid_pairs(keys, values)
    []
    """
```

Question 3

Once you were done collecting the data about the preferred names, you found out that some people have more than one preferred name! You would like to have a list of all preferred names for each person, therefore you decided to use a dictionary data structure to store this data.

Write a function that takes a list of tuples (the output from question 1, i.e. a list of (<`given_name`>, <`preferred_name`>) tuples) and converts it to a dictionary, where keys are `given_names` and values are a list of preferred names associated with that given name. Return an empty dictionary if no tuples are provided.

Notes:

1. You can assume that there aren't duplicate preferred names.
2. You can treat 'NO NAME PROVIDED' as a given name and place that in the dictionary as well.

```
def dict_of_names(name_tuples):
    """
    >>> dict_of_names([('Richard', 'Rick'),
    ... ('Roxanne', 'Rose'), ('Roxanne', 'Ann'),
    ... ('Richard', 'Ricky'), ('Roxanne', 'Roxie'),
    ... ('Mitchell', 'Mitch')])
```

```
    {'Richard': ['Rick', 'Ricky'], 'Roxanne': ['Rose', 'Ann',
'Roxie'], \
'Mitchell': ['Mitch']}

    >>> dict_of_names([('Melissa', 'Lisa'),
    ... ('Isabel', 'Bella'), ('NO NAME PROVIDED', 'Faith')])
    {'Melissa': ['Lisa'], 'Isabel': ['Bella'], \
'NO NAME PROVIDED': ['Faith']}

    >>> dict_of_names([('NO NAME PROVIDED', 'Derrick'), \
    ('NO NAME PROVIDED', 'Jacob')])
    {'NO NAME PROVIDED': ['Derrick', 'Jacob']}
    """
```

Question 4:

Your company hired three contractors to finish the painting job but the payment still needs to be decided.

Question 4.1:

Some team members suggested what they considered a fair payment and saved the information in lists with the following format:

```
[<pay for contractor 1>, <pay for contractor 2>, <pay for contractor
3>]
```

Write a function that takes a list of lists, where each inner list follows the format above, and returns a ***dictionary*** where the keys are contractor labels (e.g., '1', '2', and '3') and the values are the ***average*** payment for each contractor:

```
{'1': <average pay for contractor 1>, '2': <average pay for contractor 2>, '3':
<average pay for contractor 3>}
```

Notes:

- You can assume each inner list contains exactly 3 integers.
- If the input list is empty, the dictionary values should be 0 for all contractors.
- You don't need to define '1', '2', and '3' as magic numbers. In this class, we don't treat strings as magic numbers."
- Round to the nearest hundredth (use the `round` function).

```
def contractor_payment(suggestions):
```

```
"""
>>> contractor_payment([[10, 20, 30], [0, 20, 10]])
{'1': 5.0, '2': 20.0, '3': 20.0}

>>> contractor_payment([[10, 20, 30], [30, 20, 10], [5, 10, 15]])
{'1': 15.0, '2': 16.67, '3': 18.33}

>>> contractor_payment([[-5, -10, -4], [-20, 15, 40]])
{'1': -12.5, '2': 2.5, '3': 18.0}
"""
```

Question 4.2:

It seemed like your team members could not agree on a fair payment and some even wanted contractors to pay your company back! So you asked for a professional to help you out. The suggestion was to add a bonus pay based on their performance. To motivate them to keep delivering high-quality work, you've set up a performance-based pay formula. The bonus pay for all contractors in the next round is calculated according to the following rule:

- `bonus_pay = (`
  `0.01 * hours worked by contractor 1 +`
  `0.015 * hours worked by contractor 2 +`
  `min(0.02 * abs(100 - hours worked by contractor 3), 0.025 * hours worked by contractor 3) - 5`
  `)`
- If any contractor worked less hours than expected (represented as a negative integer), then `bonus_pay` is automatically set to -10.

Write a function that takes a dictionary of hours worked by each constructor, where the key is the constructor's label (as a string) and a value is hours worked (as an integer). This function calculates the bonus pay for all contractors based on the provided formula.

Then the function:

- Updates the dictionary with a new key-value pair: `{pay: decision}`
  - 'Bonus' if `bonus_pay` is positive
  - 'No', if `bonus_pay` is a 0
  - 'Penalty', if `bonus_pay` is negative
- Returns the calculated `bonus_pay`.

Note: You don't need to define magic numbers in the `bonus_pay` formula, but for all other parts, including the -10 value, please follow the style guide.

For example:

Input	Output	Explanation

`{`
`  '1': 200,`
`  '2': 138,`
`  '3': 172`
`}`
	`Updated dictionary:`
	`{`
	`  '1': 200,`
	`  '2': 138,`
	`  '3': 172,`
	`  'pay': 'Bonus'`
	`}`
	
	`Returns:`
	`0.51`
		Using the formula for `bonus_pay`:
		
		`0.01 * 200 +`
		`0.015 * 138 +`
		`min(0.02 * abs(100 - 172), 0.025 * 172) - 5 = 0.51`
		
		0.51 is greater than 0, so we update the dictionary with {'pay': 'Bonus'} and return 0.51

`{`
`  '1': 130,`
`  '2': 84,`
`  '3': -14`
`}`
	`Updated dictionary:`
	`{`
	`  '1': 130,`
	`  '2': 84,`
	`  '3': -14,`
	`  'pay': 'Penalty'`
	`}`
	
	`Returns:`
	`-10`
		Notice that the third contractor worked 14 less hours than expected, so according to the second rule `bonus_pay` is set to -10. -10 is less than 0, so we update the dictionary with {'pay': 'Penalty'} and return -10

```
def new_pay(hours):
    """
    >>> case1 = {'1': 200, '2': 138, '3': 172}
    >>> round(new_pay(case1), 2)
    0.51
    >>> case1
    {'1': 200, '2': 138, '3': 172, 'pay': 'Bonus'}

    >>> case2 = {'1': 130, '2': 84, '3': -14}
    >>> new_pay(case2)
    -10
    >>> case2
    {'1': 130, '2': 84, '3': -14, 'pay': 'Penalty'}

    >>> case3 = {'1': 42, '2': 96, '3': 63}
    >>> round(new_pay(case3), 1)
```

```
    -2.4
    >>> case3
    {'1': 42, '2': 96, '3': 63, 'pay': 'Penalty'}
    """
```

Question 5:

While you were busy setting up your new business, your team went out to research potential suppliers and collect samples. They visited various local markets and factories to gather a wide range of products. To keep track of their findings, they stored the information in a dictionary with the following format:

```
{‘supplier 1’: [<list of collected items>], ‘supplier 2’:[<list of collected items>], ...}
```

However, upon reviewing the items, you realized there's no need to keep duplicates. You decide to only consider *unique* items to optimize space.

Write a function that takes in a dictionary of the format described above and returns a list of **unique** items from the supplier lists, sorted **alphabetically**. If there are no items (either because the input dictionary is empty or all value lists are empty), return an empty list.

Hint:

1. You may use the built-in `sorted()` function to sort the list.
2. Nested loop might be needed. The link for the video is here for your convenience.

```
def potential_ideas_for_business(items):
    """
    >>> items = {'supplier 1': ['Tea', 'Peaches'], \
    'supplier 2': ['Peaches', 'Apples', 'Cups']}
    >>> potential_ideas_for_business(items)
    ['Apples', 'Cups', 'Peaches', 'Tea']

    >>> items = {'supplier 1': ['Flour', 'Eggs', 'Chocolate', 'Milk'], \
    'supplier 2': ['Milk', 'Eggs', 'Vanilla', 'Butter'], \
    'supplier 3': ['Butter', 'Sugar']}
    >>> potential_ideas_for_business(items)
    ['Butter', 'Chocolate', 'Eggs', 'Flour', 'Milk', 'Sugar', 'Vanilla']
```

```
    >>> items = {'supplier 1': [], 'supplier 2': []}
    >>> potential_ideas_for_business(items)
    []
    """
```

File Handling

Questions below involve reading and writing to files. Make sure that the folder called `files` is in the same directory as `hw02.py`

Question 6:

Write a function that takes a `filepath` to an non-empty file and returns the number of lines in the file.

Assumptions:

- There are no empty lines anywhere in the file
- The file always exists.

Hint: In some parts below, the built-in function [str.split()] might be useful. When using, note what the argument to this function means.

Question 6.1:

Use the `for` keyword to implement the function.

```
def count_lines_1(filepath):
    """
    >>> count_lines_1('files/test1.txt')
    6
    >>> count_lines_1('files/test2.txt')
    24
    """
```

Question 6.2:

Use the `read()` function to implement the function.

```
def count_lines_2(filepath):
    """
    >>> count_lines_2('files/test1.txt')
    6
    >>> count_lines_2('files/test2.txt')
```

```
    24
    """
```

Question 6.3:

Use the `readlines()` function to implement the function.

```
def count_lines_3(filepath):
    """
    >>> count_lines_3('files/test1.txt')
    6
    >>> count_lines_3('files/test2.txt')
    24
    """
```

Question 7:

You are given a `filepath` to a .txt file containing the name, supplier, item, and quantity of each person's findings, with each line separated by commas.

Write a function that returns a list of items in the same order as in the input file. Return an empty list if the file is empty.

Assumptions:

- There are no empty lines *if the file is not empty*
- The file always exists. (Note: An item with a quantity of 0 still counts as a collected item.)

Hints:

1. Note that when reading files, an entire line is represented as a long string.
2. To get rid of possible extra spaces or newline characters at the end of a string, consider using `str.strip()`
3. With a string of comma-separated elements, you can split it into a list and retrieve the necessary information (don't forget to use `str.split()`).
4. List indices that are not -1, 0, 1 are also considered magic numbers.

The input file format for each line is:
`name,supplier,item,quantity`

Example of an input file: (See the first doctest for output)

```
marina,vons,ice-cream,3
nicole,happy lemon,boba tea,30
```

```
charisse,petsmart,fish,2
```

```python
def collected_items(filepath):
    """
    >>> collected_items('files/ings1.txt')
    ['ice-cream', 'boba tea', 'fish']
    >>> collected_items('files/ings2.txt')
    ['shovel', 'headphones', 'bird', 'brownies']
    >>> collected_items('files/empty_trip.txt')
    []
    """
```

Question 8:

Write a function that takes in a `filepath` as a string and **writes** the number of uppercase and lowercase letters in the entire filepath to the file, in separate lines. This function should return `None`. Assume the `filepath` is always valid.

Hint:

1. The filepath is just a string, and you could manipulate/do string operations as usual.
2. The function [str.isupper()] might be helpful. Is there a similar function that checks lower case letters? Try to google it if you're not sure.
3. To get rid of extra spaces or newline characters at the end of a string, consider using [str.strip()]
4. The `filepath` provided is also the output file you should write to.

Note: The `with open...print...` in doctests opens the file you write in and checks if the content is correct. It does not hint towards the question itself.

Example: `filepath = 'files/TEST.txt'`
In the given filepath, red characters are upper and blue characters are lower, so output file should contain:

```
4
8
```

```python
def case_letters(filepath):
    """
    >>> case_letters('files/AlErNaTiNg.txt')
    >>> with open('files/AlErNaTiNg.txt', 'r') as outfile1:
    ...     print(outfile1.read().strip())
    5
```

```
13
>>> case_letters('files/another_test.txt')
>>> with open('files/another_test.txt', 'r') as outfile2:
...    print(outfile2.read().strip())
0
19
"""
```

Question 9:

Now it's time to combine reading from and writing to files in the same function. Each person wrote down their office number (as an integer), and we need to simplify this information for the general public.

Given a filepath to a .txt file containing n lines (where n>0), with an office number on each line, write a function that:

Maps each office number to a new file called `files/floors.txt`, with each mapping on a new line, follow these mapping rules:

a. Less than 1 is '`not a valid office number`'
b. Between 1 and 199 (inclusive) is '`ground floor`'
c. Between 200 and 299 (inclusive) is '`second floor`'
d. Above 300 (inclusive) is '`third floor and above`'

2. Returns the sum of all **valid** office numbers, i.e. ignoring the non-positive ones.

Assumptions:

1. There is an integer on every line and the file will not be empty.
2. Assume the file always exists.
3. You will always write to the `files/floors.txt` file, and you don't need to remove the last new line in this output file.

Hints:

1. What is the data type of a line when reading it in?
2. To remove extra spaces or newline characters from the end of a string, consider using `str.strip()` before processing the number.

Example:

Input File `files/offices1.txt`:

```
25
-3
234
```

Output File `files/floors.txt`:

```
ground floor
not a valid office number
second floor
```

Return Value: 25 + 234 = 259

```python
def map_office(filepath):
    """
    >>> map_office('files/offices1.txt')
    259
    >>> with open('files/floors.txt', 'r') as f:
    ...    print(f.read().strip())
    ground floor
    not a valid office number
    second floor

    >>> map_office('files/offices2.txt')
    734
    >>> with open('files/floors.txt', 'r') as f:
    ...    print(f.read().strip())
    third floor and above
    not a valid office number
    second floor
    ground floor
    """
```

Submission

When submitting to Gradescope, you can include the files folder by compressing it with your hw02.py file, creating a .zip file you can submit that zip file to Gradescope. If you have your hw02.py and files folder inside of another folder, make sure that you select hw02.py and the files folder directly to compress it, rather than compressing the encompassing folder, as it will cause issues with your submission.

You can do this by going into File Explorer or Finder and selecting **both** the files folder and the hw02.py file. Right-click and then select either

Windows: Send to -> Compressed (zipped) folder
Mac: Compress 2 items

Your .zip file should have the following files if you open it:

```
files/
--- offices1.txt
--- offices2.txt
--- ... (other files)
hw02.py
```

Please submit the homework via Gradescope. You may submit more than once before the deadline, and only the final submission will be graded. Please refer to the Gradescope FAQ if you have any issues with your submission.